

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 2 – Pseudocode Writing Task (Algorithm Design)

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO5 – Articulation	Assessment:	Assessment Tool 2 – Pseudocode Writing Task (Algorithm Design)
Weightage:	35% of CIA		
CO5 Statement:	Implement semantic models using predicate logic, word sense disambiguation and validate information retrieval techniques. (BL-5)		
Student Name:	S.Muskan	Reg. No.:	192472182
Section / Batch:	BE-CSE-AI	Date:	

Code of Conduct

*I **S.Muskan(192472182)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: **S.Muskan**

Instructions

- Draw necessary diagrams such as constraint graphs, coreference chains, discourse trees, or predicate logic representations wherever applicable.
- Generate solutions that fully satisfy all given constraints and provide clear evaluation of the output.
- Answers should be concise, well-structured, and supported by evidence from the given text/scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Reference Resolution, Discourse	Entity Linking Algorithm	Q1
Text Coherence, Discourse Structure	Coherence Analysis Algorithm	Q2
Dialog Acts, Conversational Agents	Intent Recognition Algorithm	Q3
Language Generation, Surface Realization	NLG Pipeline Design	Q4
Machine Translation, Interlingua, Statistical Approaches	Translation Workflow Algorithm	Q5

Annexure A – Pseudocode Writing Task (Algorithm Design)

1. Design an algorithm to perform **Reference Resolution** by identifying pronouns and linking them to the correct entities in a discourse. Apply your algorithm to the text:

"Ravi met Arun at the library. He borrowed a book and later returned it."

Generate the resolved discourse by replacing pronouns with their corresponding entities. Further, validate the correctness of the resolution process by explaining how discourse context is used to identify the antecedents.

2. Develop an algorithm to analyze **Text Coherence** and **Discourse Structure** by identifying logical relationships between sentences such as cause-effect, sequence, and elaboration. Apply your algorithm to the following discourse:

"The roads were flooded after heavy rainfall. Therefore, schools were closed for the day. Students attended classes online."

Identify the discourse relations among the sentences and construct a discourse structure representation. Further, justify how these relations contribute to maintaining coherence in the text.

3. Design an algorithm for a **Conversational Agent** that identifies **Dialogue Acts** from user interactions. Apply your algorithm to the conversation:

User: "Can you book a train ticket for me?" **Agent:**

"Sure, where would you like to travel?" **User:** "I want to go to Chennai."

Agent: "Your ticket has been booked."

Classify each utterance as Request, Question, Inform, Confirmation, or Action. Generate the dialogue-act sequence and evaluate how the identified acts help the conversational agent interpret user intentions.

4. Design an algorithm for **Language Generation** that converts a semantic representation into a grammatically correct sentence through **Surface Realization**. Consider the semantic input:

Action: Buy
Agent: Student
Object: Book
Tense: Past

Generate the final natural language sentence and explain how the algorithm performs lexical selection, sentence structuring, and surface realization. Further, validate the grammatical correctness of the generated output.

5. Design an algorithm that combines **Interlingua-based Machine Translation** and **Statistical Translation Approaches**. Apply your algorithm to translate the sentence:

"The boy is playing football."

Perform the following steps:

1. Analyze the source sentence.
2. Create an Interlingua representation.

3. Generate candidate target-language translations.
4. Apply statistical scoring to select the most appropriate translation.
5. Produce the final translated sentence.

Finally, evaluate how the Interlingua representation and statistical methods improve translation quality and reduce ambiguity.

Q1. Reference Resolution – Entity Linking Algorithm

Problem Understanding

Input:

"Ravi met Arun at the library. He borrowed a book and later returned it."

Output:

The pronouns must be linked to their correct antecedents.

Pronouns:

- **He** → person/entity
- **it** → object/entity

Constraints:

1. Gender agreement
2. Number agreement
3. Recency
4. Grammatical compatibility
5. Discourse coherence

Coreference Analysis

The first sentence introduces two possible male antecedents:

Ravi —————
|————> possible antecedent for "He"
Arun —————

Since **Arun** is the most recent compatible male entity, the recency constraint selects:

He → Arun

For **it**, the candidates include entities previously mentioned, but **book** is the only suitable singular inanimate object.

book → it

Coreference Chain

Ravi —→ Ravi

Arun —→ He —————→ Arun

book —→ it —————→ book

Pseudocode

ALGORITHM ReferenceResolution(Text)

INPUT:

Text containing sentences and referring expressions

OUTPUT:

Resolved text with pronouns replaced by antecedents

BEGIN

Sentences $\leftarrow$ SplitIntoSentences(Text)

EntityList $\leftarrow$ EmptyList

ResolvedText $\leftarrow$ EmptyList

FOR each Sentence in Sentences DO

Mentions $\leftarrow$ IdentifyEntitiesAndPronouns(Sentence)

FOR each Mention in Mentions DO

IF Mention is a new entity THEN

Add Mention to EntityList

ELSE IF Mention is a pronoun THEN

Candidates $\leftarrow$ FindPreviousEntities(EntityList)

FOR each Candidate in Candidates DO

Score[Candidate] $\leftarrow$ 0

IF GenderCompatible(Mention, Candidate) THEN

Score[Candidate] $\leftarrow$ Score[Candidate] + 2

END IF

IF NumberCompatible(Mention, Candidate) THEN

Score[Candidate] $\leftarrow$ Score[Candidate] + 2

END IF

IF SemanticallyCompatible(Mention, Candidate) THEN

Score[Candidate] $\leftarrow$ Score[Candidate] + 2

END IF

IF Candidate is MostRecent THEN

Score[Candidate] $\leftarrow$ Score[Candidate] + 1

END IF

END FOR

BestCandidate $\leftarrow$ CandidateWithHighestScore()

Replace Mention with BestCandidate

END IF

END FOR

Add Sentence to ResolvedText

END FOR

RETURN ResolvedText

END

Application to Given Text

Input:

Ravi met Arun at the library.

He borrowed a book and later returned it.

↓

He → Arun

it → book

↓

Resolved:

Ravi met Arun at the library.

Arun borrowed a book and later returned the book.

Validation

The resolution is valid because:

- **He → Arun:** Arun is the nearest compatible male antecedent.
- **it → book:** “book” is an inanimate singular object and fits the verb “returned.”
- The resulting entity chain remains grammatically and semantically coherent.

Final Answer:

“Ravi met Arun at the library. Arun borrowed a book and later returned the book.”

Q2. Text Coherence and Discourse Structure Algorithm

Given Discourse

“The roads were flooded after heavy rainfall. Therefore, schools were closed for the day. Students attended classes online.”

Problem Understanding

The algorithm must identify relationships between sentences, particularly:

- Cause
- Effect
- Sequence

Discourse Analysis

Heavy rainfall



Roads flooded



Schools closed

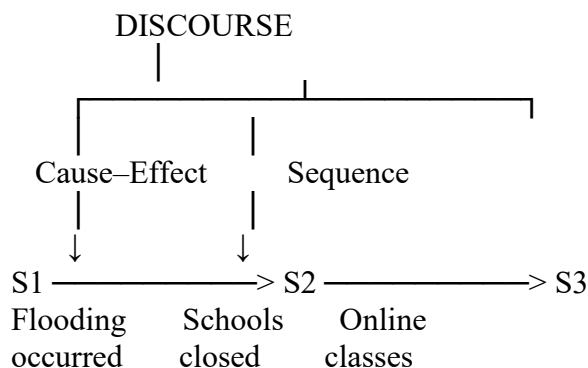


Students attended classes online

Therefore:

- S1 → S2: Cause–Effect
- S2 → S3: Sequence / Consequence

Discourse Structure



Pseudocode

ALGORITHM CoherenceAnalysis(Text)

INPUT:

A sequence of sentences

OUTPUT:

Discourse relations and coherence structure

BEGIN

```
Sentences ← SplitIntoSentences(Text)

FOR i ← 1 TO Length(Sentences) - 1 DO

    Current ← Sentences[i]
    Next ← Sentences[i + 1]

    Relation ← NONE

    IF ContainsCauseMarker(Current, Next)
        OR IndicatesCauseEffect(Current, Next) THEN

        Relation ← "CAUSE-EFFECT"

    ELSE IF ContainsSequenceMarker(Current, Next)
        OR IndicatesTemporalOrder(Current, Next) THEN

        Relation ← "SEQUENCE"

    ELSE IF IndicatesExplanation(Current, Next) THEN

        Relation ← "ELABORATION"

    ELSE

        Relation ← "NO EXPLICIT RELATION"

    END IF

    StoreRelation(Current, Next, Relation)

END FOR

BuildDiscourseTree()

IF AllImportantSentencesAreConnected() THEN
    Coherent ← TRUE
ELSE
    Coherent ← FALSE
END IF

RETURN Relations, DiscourseTree, Coherent

END
```

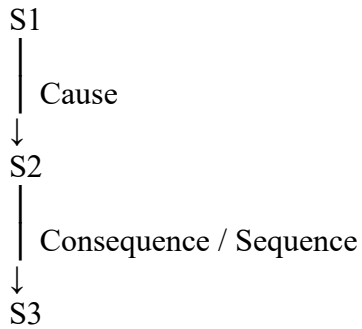
Application

S1 = Roads were flooded after heavy rainfall.

S2 = Therefore, schools were closed for the day.

S3 = Students attended classes online.

Analysis:



Why the Text Is Coherent

The sentences form a logical chain:

Heavy Rainfall→Flooded Roads→School Closure→Online Classes

The word “**Therefore**” explicitly signals the relationship between flooding and school closure.

The third sentence logically follows the closure because students switch to online classes.

Validation

- Condition 1: Logical connection ✓
- Condition 2: Cause-effect relation ✓
- Condition 3: Sequential flow ✓
- Condition 4: Topic continuity ✓

Result: COHERENT

Conclusion: The discourse is coherent because each sentence contributes to the same event chain and the relationships between events are logically understandable.

Q3. Dialogue Act and Intent Recognition Algorithm

Given Conversation

User: “Can you book a train ticket for me?”

Agent: “Sure, where would you like to travel?”

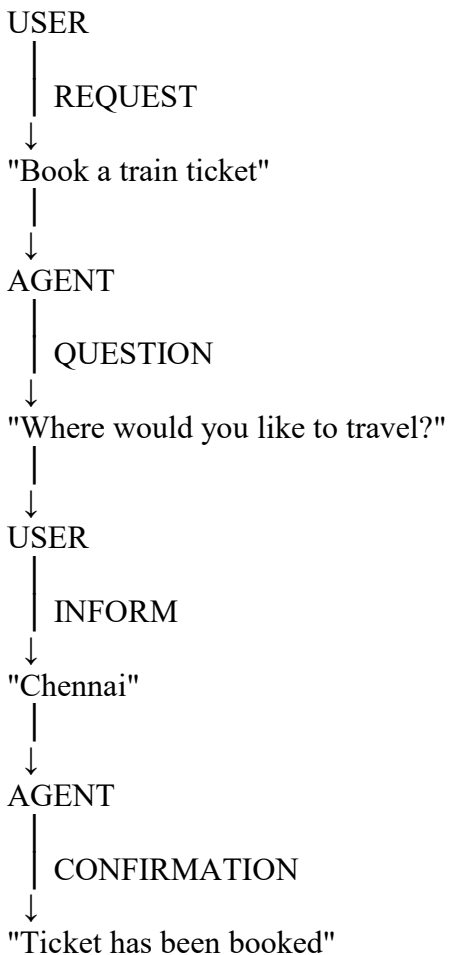
User: “I want to go to Chennai.”

Agent: “Your ticket has been booked.”

Dialogue-Act Classification

Speaker	Utterance	Dialogue Act
User	Can you book a train ticket for me?	Request
Agent	Sure, where would you like to travel?	Question
User	I want to go to Chennai.	Inform
Agent	Your ticket has been booked.	Confirmation

Dialogue Sequence



Pseudocode

ALGORITHM IntentRecognition(Utterance)

INPUT:

User or agent utterance

OUTPUT:

Dialogue Act

BEGIN

Text $\leftarrow$ Normalize(Utterance)

IF ContainsBookingRequest(Text)

OR ContainsWords(Text, ["book", "reserve"]) THEN

Act $\leftarrow$ "REQUEST"

ELSE IF ContainsQuestionPattern(Text)

OR EndsWithQuestionMark(Text) THEN

Act $\leftarrow$ "QUESTION"

ELSE IF ContainsConfirmationPattern(Text)

OR ContainsWords(Text, ["confirmed", "booked", "done"]) THEN

Act $\leftarrow$ "CONFIRMATION"

ELSE IF ContainsInformation(Text)

OR ContainsDestination(Text)

OR ContainsPreference(Text) THEN

Act $\leftarrow$ "INFORM"

ELSE IF ContainsActionCommand(Text) THEN

Act $\leftarrow$ "ACTION"

ELSE

Act $\leftarrow$ "UNKNOWN"

END IF

RETURN Act

END

Applying the Algorithm

"Can you book a train ticket for me?"

↓

REQUEST

"Sure, where would you like to travel?"



QUESTION

"I want to go to Chennai."



INFORM

"Your ticket has been booked."



CONFIRMATION

Intent Interpretation

The sequence allows the agent to understand the user's intention progressively:

REQUEST



Need for ticket booking



QUESTION



Destination required



INFORM



Destination = Chennai



CONFIRMATION



Booking completed

Validation

The dialogue acts are correctly ordered and mutually consistent.

Request → Question → Inform → Confirmation



Conclusion: Dialogue-act recognition allows the conversational agent to determine what the user wants, identify missing information, collect it, and confirm completion.

Q4. Natural Language Generation – Surface Realization

Semantic Input

Action : Buy

Agent : Student

Object : Book

Tense : Past

Problem Understanding

The NLG system must convert the structured semantic representation into a grammatically correct English sentence.

Expected output:

“The student bought a book.”

NLG Pipeline

Semantic Representation



Lexical Selection



Sentence Planning



Word Ordering



Morphological Realization



Surface Sentence

Pseudocode

ALGORITHM SurfaceRealization(SemanticInput)

INPUT:

Action, Agent, Object, Tense

OUTPUT:

Grammatically correct sentence

BEGIN

AgentWord ← SelectLexeme(Agent)

ObjectWord ← SelectLexeme(Object)

ActionWord ← SelectLexeme(Action)

IF Tense = "Past" THEN

 ActionWord ← ConvertToPastTense(ActionWord)

END IF

```
AgentPhrase ← AddDeterminer(AgentWord)
ObjectPhrase ← AddDeterminer(ObjectWord)
```

```
SentenceStructure ←
  [AgentPhrase, ActionWord, ObjectPhrase]
```

```
Sentence ← ArrangeWords(SentenceStructure)
```

```
Sentence ← ApplyCapitalization(Sentence)
```

```
Sentence ← AddFullStop(Sentence)
```

```
IF CheckGrammar(Sentence) = TRUE THEN
```

```
  RETURN Sentence
```

```
ELSE
```

```
  RETURN "Generation failed"
```

```
END IF
```

```
END
```

Step 1 – Lexical Selection

Agent → Student

Action → Buy

Object → Book

Step 2 – Tense Realization

The input specifies **Past tense**.

Buy

↓

Bought

Step 3 – Sentence Structure

English follows:

Subject + Verb + Object

Therefore:

Student + bought + book

Step 4 – Determiner Selection

The generated noun phrases become:

The student

a book

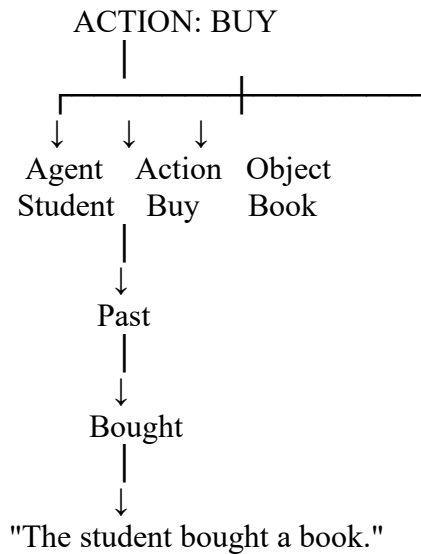
Step 5 – Final Surface Realization

The student + bought + a book.

↓

"The student bought a book."

Semantic-to-Surface Diagram



Grammatical Validation

Property	Result
Subject present	✓
Verb present	✓
Object present	✓
Past tense	✓
Subject–verb structure	✓
Correct word order	✓
Complete sentence	✓
Punctuation	✓

Final Generated Sentence:

The student bought a book.

Conclusion: The NLG pipeline correctly converts the semantic representation into a grammatically valid surface sentence by performing lexical selection, tense realization, sentence structuring, and grammatical validation.

Q5. Interlingua + Statistical Machine Translation Workflow

Source Sentence

“The boy is playing football.”

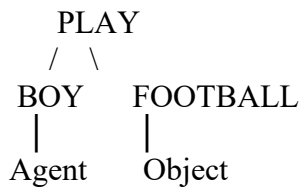
For this application, **Tamil** is selected as the target language.

Step 1 – Source Sentence Analysis

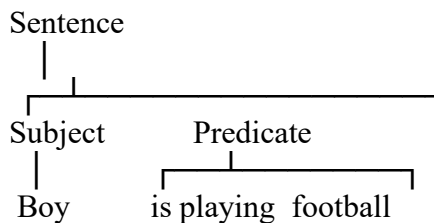
Token-Level Analysis

Word	Role
The boy	Subject
is playing	Present progressive action
football	Object

Semantic structure:



Syntactic Representation



Step 2 – Interlingua Representation

The interlingua should preserve the meaning independently of the target language.

EVENT:

Action = PLAY
Agent = BOY
Object = FOOTBALL
Tense = PRESENT
Aspect = PROGRESSIVE
Number = SINGULAR
Definiteness = DEFINITE

Predicate Representation

Boy(x)
Football(y)
Playing(x,y)
Present(x,y)

Progressive(x,y)

Combined:

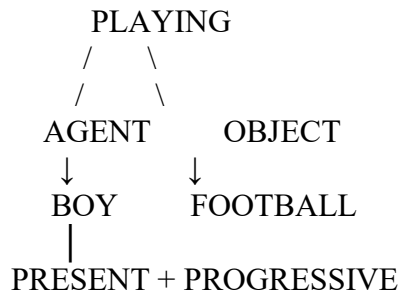
Boy(x) $\wedge$ Football(y)

$\wedge$ Playing(x,y)

$\wedge$ Present(x,y)

$\wedge$ Progressive(x,y)

Interlingua Diagram



Step 3 – Candidate Target Translations

Possible Tamil candidates:

Candidate 1

“ .”

Meaning:

The boy is playing football.

Candidate 2

“ .”

This expresses the action but does not represent the progressive aspect as explicitly as Candidate 1.

Candidate 3

“ .”

This can be understood, but the object marking and naturalness are less suitable for this context.

Step 4 – Statistical Scoring

A statistical translation module can score each candidate using features such as:

Score =

$\lambda_1 \times \text{SemanticSimilarity}$

+

$\lambda_2 \times \text{LanguageModelScore}$

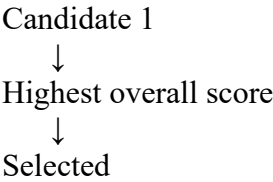
+

$\lambda_3 \times \text{GrammarScore}$
+
 $\lambda_4 \times \text{ContextScore}$

For illustration:

Candidate	Semantic Match	Grammar	Fluency	Overall
Candidate 1	High	High	High	Best
Candidate 2	High	High	Medium	Good
Candidate 3	Medium	Medium	Medium	Lower

Therefore:



Step 5 – Final Translation

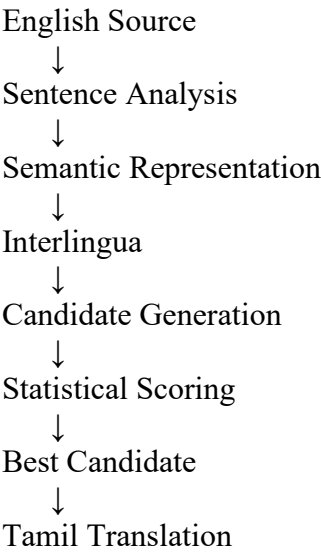
English

The boy is playing football.

Tamil

.

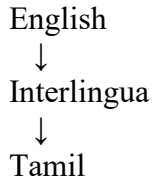
Translation Workflow



Evaluation

Role of Interlingua

Interlingua separates **meaning from language-specific surface form**.



This helps preserve:

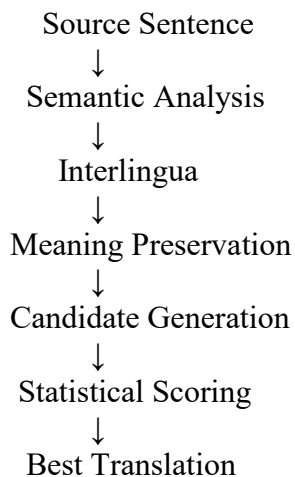
- Agent = boy
- Action = playing
- Object = football
- Present tense
- Progressive aspect

Role of Statistical Translation

Statistical scoring helps select the candidate that best satisfies:

- Semantic similarity
- Grammatical correctness
- Language fluency
- Contextual appropriateness

Combined Approach



Advantages

Approach	Main Advantage
Interlingua	Preserves language-independent meaning
Statistical scoring	Selects the most probable fluent output
Combined approach	Reduces ambiguity while maintaining natural translation

Conclusion: The combined approach is stronger than using either technique alone. Interlingua protects the semantic structure, while statistical scoring helps select the most appropriate target-language realization.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q. No. / Criteria Level	Problem Understanding	Algorithm Design	Use of Control Structures	Pseudocode Structure	Completeness	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____